

일정 입력을 다시 짜기

지금의 일정 편집 패널은 하나를 넣는 데 **클릭 11번**이 필요하다. 시중 앱 8종이 이 문제를 어떻게 풀었는지 뜯어보고, TomaDesk에 맞는 입력 모델과 외부 캘린더 연동 경로를 단계별로 정리했다.

대상 축 2 · 캘린더 / 일정 편집기

스택 Python 3.12 · PyQt6 · SQLite

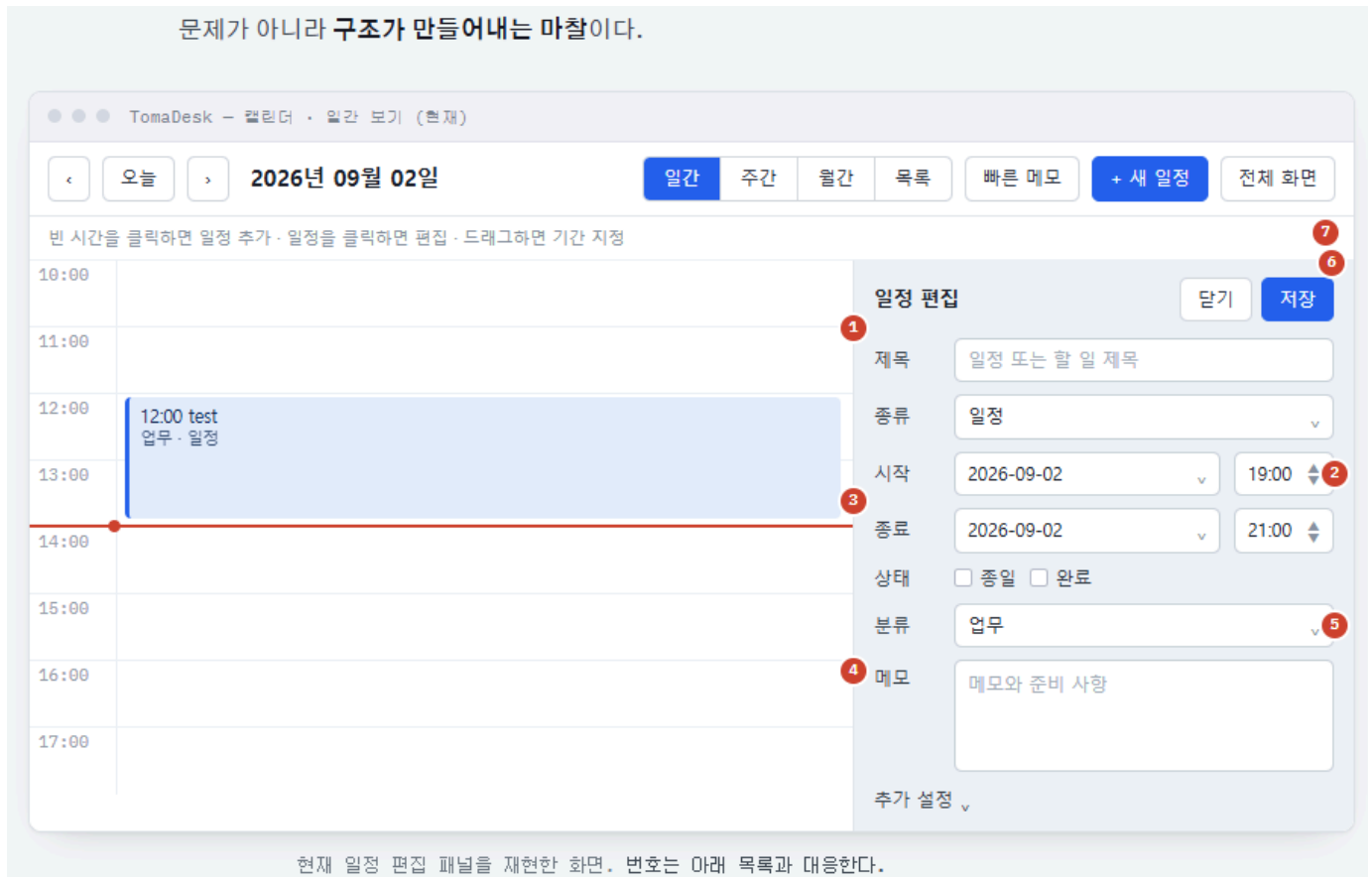
작성 2026-09-02

1. [01현재 방식 진단](#)
2. [02시중 앱 입력 패턴](#)
3. [03캘린더 연동](#)
4. [04개선 설계](#)
5. [05로드맵](#)
6. [06결정할 것](#)

01 — 진단

지금 화면에서 실제로 일어나는 일

캡처한 화면을 그대로 재현하고, 입력 흐름이 끊기는 지점에 번호를 붙였다. 기능이 없어서 생기는 문제가 아니라 **구조가 만들어내는 마찰**이다.



현재 일정 편집 패널을 재현한 화면. 번호는 아래 목록과 대응한다.

1. 1

패널이 화면의 27%를 상시 점유한다. 일정 하나를 만드는 동안 캘린더의 오른쪽 절반이 가려진다. 새 일정을 넣는 이유는 대개 "다른 일정과 겹치나?"인데, 정작 그때 캘린더가 좁아진다.

2. 2

날짜·시간이 4개의 컨트롤로 쪼개져 있다. 시작 날짜 콤보 + 시각 스피너, 종료 날짜 콤보 + 시각 스피너. 설명서대로 20260910 · 930 직접 입력이 되더라도, 보이는 형태가 드롭다운이라 사용자는 타이핑을 시도하지 않는다. 기능이 있어도 발견되지 않는다.

3. 3

기본값이 맥락과 무관하다. 12:00 근처를 보고 있고 현재 시각이 13:40인데 폼은 19:00~21:00을 제시한다. 클릭한 시간·보고 있는 시간이 반영되지 않으면 사용자는 매번 두 필드를 고쳐야 한다.

4. 4

메모 입력란이 폼 면적의 3분의 1이다. 실제 사용 빈도(대부분 비워 둠)에 비해 시각적 비중이 과하다. 위계가 사용 빈도를 따라가지 않는다.

5. 5

분류가 무채색 드롭다운이다. 캘린더의 일정 칩은 카테고리 색으로 구분되는데, 정작 색을 정하는 자리에는 색이 없다. 결과를 보기 전까지 무슨 색인지 알 수 없다.

6. 6

저장 버튼이 흐름의 시작 위치에 있다. 입력은 위에서 아래로 진행되는데 확정 버튼은 맨 위 오른쪽이다. 마지막 필드에서 Enter로 저장되지 않으면 마우스가 화면을 되돌아 올라가야 한다.

7. 7

조작법을 상시 텍스트로 설명한다. "빈 시간을 클릭하면..."이 항상 떠 있다는 것은 조작이 스스로 드러나지 않는다는 뜻이다. 설명서 9장의 "빈 것은 보여주지 않는다" 원칙과도 어긋난다.

일정 하나 = 클릭 11회 · 필드 8개 · 시선 이동 4회. 같은 일정이 Fantastical에서는 단축키 1 + 타이핑 1줄 + Enter다. 측정 기준: "내일 오후 3시 팀 회의 · 업무" 입력

02 — 조사

시중 앱은 다섯 가지 방식으로 푼다

일정 입력 UI는 지난 10년간 다섯 갈래로 수렴했다. 어느 하나가 정답이 아니라, 성숙한 앱은 같은 일정을 세 가지 이상의 경로로 넣을 수 있게 두고 사용자가 상황에 맞게 고르게 한다.

A. 자연어 한 줄 — Fantastical · Todoist · Google 빠른 추가

말하듯이 한 줄을 치면 제목·날짜·시간·반복·분류가 분해된다. Fantastical은 캘린더를 /업무, 참석자를 with, 장소를 at 으로 받고, 파싱된 결과를 입력하는 동안 미리보기로 보여준다. 미리보기 자체를 다른 시간대로 드래그하면 입력값이 따라 바뀐다.

Todoist는 여기에 두 가지를 더한다. 인식된 부분에 밑줄이 그어져 무엇이 날짜로 잡혔는지 타이핑 중에 보이고, 잘못 잡힌 단어(예: "월간 보고서"의 "월간")를 클릭하면 일반 텍스트로 되돌아간다. 파서가 틀릴 수 있다는 전제를 UI에 그대로 반영한 설계다.

되돌아간다. 파서가 틀릴 수 있다는 전제를 UI에 그대로 반영한 설계다.

● ● ● 자연어 한 줄 입력 - 인식 하이라이트 + 결과 미리보기

9월 10일

14시~16시

월간보고

#업무

!30분전

월간보고

2026-09-10 (목)

14:00 - 16:00

업무

30분 전 알림

반복 없음

Enter 저장 Tab 상세 편집 Esc 취소

인식된 단어를 클릭하면 일반 텍스트로 되돌립니다

Fantastical의 실시간 미리보기와 Todoist의 인식 하이라이트 + 클릭 취소를 합친 형태. 색은 필드 종류를 뜻한다 - 파랑 날짜, 초록 시간, 빨강 분류, 주황 알림.

B. 캔버스 직접 조작 - Google · Apple · Outlook 캘린더

Fantastical의 실시간 미리보기와 Todoist의 인식 하이라이트 + 클릭 취소를 합친 형태. 색은 필드 종류를 뜻한다 - 파랑 날짜, 초록 시간, 빨강 분류, 주황 알림.

B. 캔버스 직접 조작 - Google · Apple · Outlook 캘린더

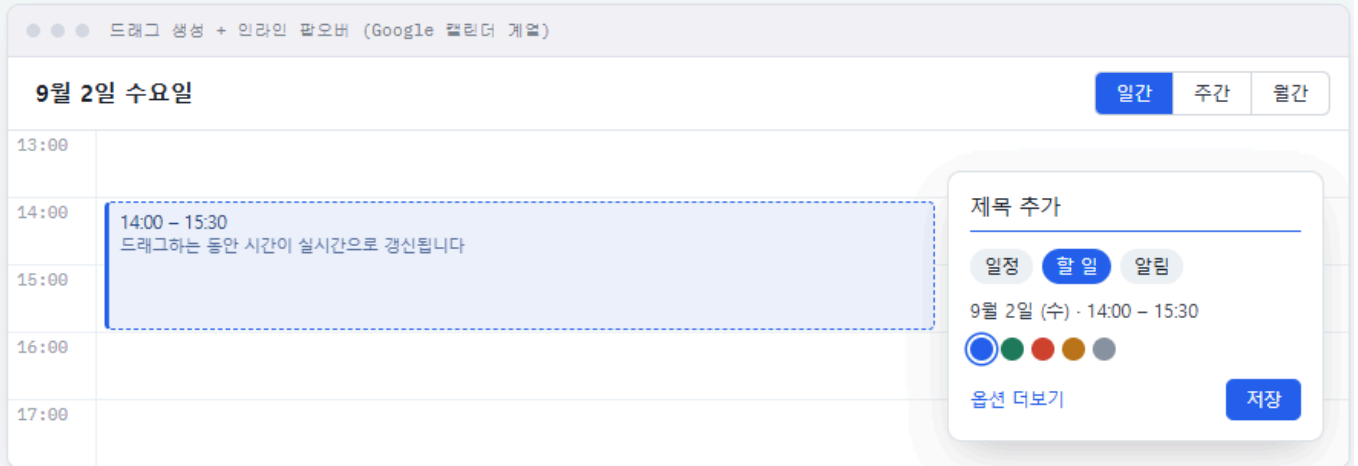
시간축 위에서 **드래그가 곧 시간 입력**이다. 빈 곳을 끌어 내리면 그 범위가 시작·종료가 되고, 만들어진 뒤에도 블록을 옮기면 시간이 바뀌고 아래쪽 모서리를 잡아 늘이면 길이가 바뀐다. 날짜·시간 필드를 한 번도 건드리지 않고 일정이 완성된다.

핵심은 **결과가 즉시 제자리에 보인다**는 점이다. 폼에 19:00을 입력하고 저장한 뒤 캘린더에서 확인하는 것과, 19시 칸을 끌어 만드는 것은 확인 비용이 다르다.

C. 인라인 팝오버 - 클릭한 자리에서 끝낸다

Google 캘린더는 빈 칸을 클릭하면 **그 자리에 작은 버블**이 뜬다. 제목만 치고 저장하면 끝이고, 반복·참석자·알림이 필요하면 "옵션 더보기"로 전체 편집 화면으로 승격된다. **대다수의 일정은 제목과 시간뿐**이라는 사실을 UI 구조로 인정한 것이다.

제목과 시간뿐이라는 사실을 UI 구조로 인정한 것이다.



빈 칸을 끌면 점선 블록이 따라오고, 손을 떼면 그 자리에 팝오버가 뜬다. 분류는 드롭다운이 아니라 색 점이고, 나머지는 "옵션 더보기" 뒤로 접힌다.

D. 키보드 우선 / 커맨드 팔레트 — Notion Calendar · Vimcal

빈 칸을 끌면 점선 블록이 따라오고, 손을 떼면 그 자리에 팝오버가 뜬다. 분류는 드롭다운이 아니라 색 점이고, 나머지는 "옵션 더보기" 뒤로 접힌다.

D. 키보드 우선 / 커맨드 팔레트 — Notion Calendar · Vimcal

마우스를 아예 쓰지 않는 층을 따로 둔다. Notion Calendar는 ? 하나로 전체 단축키 목록을 띄우고, 시스템 전역 단축키를 설정에서 지정하게 한다. Vimcal은 Cmd+K 커맨드 팔레트에서 일정 생성부터 템플릿 적용까지 전부 처리한다.

이 패턴은 TomaDesk와 가장 궁합이 좋다. 이미 전역 단축키 엔진(축 1)과 **Ctrl+Alt+N** 빠른 메모라는 검증된 선례가 있다. 빠른 일정이 없을 뿐이다.

E. 리스트에서 캘린더로 드래그 — TickTick · Sunsama · Motion

왼쪽 할 일 목록에서 항목을 집어 시간축에 떨어뜨리면 그 시간에 배치된다(타임 블로킹). 할 일과 일정을 같은 화면 안에서 **오가게** 만드는 방식이다.

TomaDesk에는 이미 이 재료가 있다 — **메모 목록**과 **D-Day**. "이 메모를 목요일 오후에 하겠다"를 드래그로 표현할 수 있고, 그렇게 만들어진 일정은 원본 메모와 연결되어 알림·포스트잇까지 그대로 물린다.

입력 방식 비교 — TomaDesk 적용 관점				
앱주 입력 방식특징적인 장치PyQt6 난이도도도입 가치				
Fantastical	자연어 한 줄 + 미니창	타이핑 중 실시간 미리보기, 미리보기를 드래그해 시간 조정	중	최우선
Todoist	쿼에드 Q	인식 부분 밑줄, 클릭하면 인식 취소 , # 프로젝트 ! 알림 + 담당자	중	최우선
Google 캘린더	드래그 생성 + 인라인 팝오버	제목만으로 저장 가능, "옵션 더보기"로 승격, 색 점으로 분류 선택	중	최우선
Apple 캘린더	드래그 생성 · Siri 음성	블록 이동/리사이즈로 시간 수정, 폼 없이 완결	중	높음
Notion Calendar	키보드 우선	? 단축키 오버레이, 시스템 전역 단축키 설정	낮음	높음
Vimcal	커맨드 팔레트	Cmd K에서 생성 · 템플릿 · 설정까지 일원화	중	중간
TickTick	리스트 → 타임라인 드래그	할 일과 일정이 한 화면, 주간 뷰에서 타임 블로킹	높음	중간
Sunsama / Motion	리스트 → 타임라인 드래그	하루 계획 의식(ritual) 중심, 미배치 항목을 남겨두지 않음	높음	참고

조사에서 나온 한 가지 경고

2026년 기준으로 자연어 입력을 **생성에만** 쓰는 앱이 대부분이다. "그 회의 30분 뒤로 밀어줘" 같은 수정은 거의 어느 앱도 자연어로 처리하지 못한다. TomaDesk도 1단계에서는 **생성 전용**으로 못 박고, 수정은 캔버스 드래그(B)에 맡기는 편이 낫다. 한 번에 둘 다 노리면 파서가 감당하지 못한다.

외부 캘린더와 어떻게 물릴 것인가

연동은 전부-아니면-전무가 아니다. 난이도와 위험이 다른 네 개의 층이 있고, 아래층부터 쌓아 올리면 각 단계가 그 자체로 쓸모 있다.

연동 방식 4단계					
충방식방향대상 구현난이도					
L1	.ics 내보내기 / 가져오기	수동 · 양방향	모든 캘린더 앱	icalendar 패키지, 표준 라이브러리로도 가능	1일
L2	.ics 구독 URL 읽기	단방향 (읽기)	회사 캘린더, 공휴일, 구글 공개 캘린더	requests + icalendar, 주기적 폴링	2일
L3	CalDAV	양방향	iCloud, Nextcloud, Fastmail, 네이버	caldav 패키지, 앱 암호 방식이라 OAuth 불필요	1주
L4	공식 API	양방향 · 증분	Google Calendar, Outlook / Microsoft 365	Google syncToken, Graph delta 쿼리	2-3 주
L0	로컬 Outlook (COM)	양방향	사내망 · 폐쇄망 PC	pywin32 의 win32com.client, 인증 불필요	3일

연동을 설계할 때 반드시 정해야 하는 네 가지

- ID 매핑 테이블.** external_events(local_id, provider, external_id, etag, last_synced, deleted_at). TomaDesk의 일정 테이블에 외부 ID 컬럼을 얹지 말고 별도 테이블로 두어야, 한 일정이 두 캘린더에 동시에 연결되는 경우를 감당한다.
- 증분 동기화.** 전체 재조회는 금물이다. Google은 syncToken, Microsoft Graph는 delta 링크를 돌려준다. 토큰이 만료되면 서버가 410 Gone 을 주고, 그때만 전체 재동기화를 한다.
- 충돌 정책.** 로컬 우선 / 원격 우선 / 사용자 선택 중 하나를 고정해 두고 설정으로 뺀다. 설명서 9장 원칙 5("애매한 것은 설정으로 넘긴다")가 그대로 적용된다.
- 읽기 전용 미러부터.** 외부 캘린더를 회색 배경 블록으로만 표시하고 편집을 막는 단계를 반드시 거친다. 양방향 쓰기를 처음부터 켜면, 버그 하나가 사용자의 실제 업무 캘린더를 훼손한다.

이 프로그램에는 L1 → L0/L3 → L4 순서를 권한다

TomaDesk는 데스크톱 상주 프로그램이고 이미 Excel 내보내기/가져오기가 있다. .ics 왕복(L1)은 그 옆자리에 하루면 불고, 그것만으로 "구글 캘린더에 옮기고 싶다"는 요구의 상당수가 해소된다. OAuth 클라이언트 등록·토큰 보관·리프레시가 필요한 L4는, 사용자가 실제로 요구할 때 착수해도 늦지 않다.

TomaDesk에 맞는 입력 모델

조사한 다섯 방식을 그대로 옮기는 게 아니라, 이 프로그램이 이미 가진 것 위에 얹는다. 전역 단축키 엔진, 빠른 메모의 선택, 메모-D-Day-알림이 물려 있는 구조가 그 재료다.

① 빠른 일정 — Ctrl+Alt+A

빠른 메모(Ctrl+Alt+N)의 **일정 판**이다. 어느 프로그램에 있던 한 줄을 치고 Enter를 누르면 일정이 생긴다. TomaDesk의 설계 의도인 "손을 떼지 않는 것"에 정확히 부합하고, 창을 띄우는 비용이 0이다.

비용이 0이다.

빠른 일정 - 전역 단축키로 뜨는 한 줄 창

빠른 일정 · CTRL+ALT+A

다음주 화요일 10시 치과 #개인

치과

2026-09-08 (화)

10:00 - 11:00

개인

알림 없음

같은 시간대에 다른 일정 없음

Enter 저장 Tab 상세 D D-Day로 M 메모로 Esc 취소

미리보기에 충돌 검사 한 줄을 넣었다. 일정을 넣는 진짜 이유가 "겹치나?"이기 때문이다. D·M 키로 같은 입력을 D-Day나 메모로 돌릴 수 있게 하면, 이 창 하나가 속 2 전체의 입구가 된다.

② 한국어 파서 — 라이브러리로 안 된다

먼저 확인부터 했다. 파이썬에서 가장 널리 쓰이는 `dateparser` 에 한국어를 넣어 보면 이렇다.

미리보기에 충돌 검사 한 줄을 넣었다. 일정을 넣는 진짜 이유가 "겹치나?"이기 때문이다. D·M 키로 같은 입력을 D-Day나 메모로 돌릴 수 있게 하면, 이 창 하나가 속 2 전체의 입구가 된다.

② 한국어 파서 — 라이브러리로 안 된다

먼저 확인부터 했다. 파이썬에서 가장 널리 쓰이는 `dateparser` 에 한국어를 넣어 보면 이렇다.

dateparser 4.x · languages=['ko'] 실측		
입력결과판정		
내일 오후 3시	2026-09-03 08:12 (현재 시각 그대로)	시각 무시
다음주 화요일 10시	None	실패
9월 10일 14시	None	실패
모레 아침 9시	None	실패
금요일 저녁 7시	None	실패
매주 월요일	None	실패

결론은 명확하다. **규칙 기반 파서를 직접 짠다**. 한국어 일정 표현은 어휘가 좁고(오늘·내일·모레·요일·N시·오전/오후·매주·매달) 어순이 안정적이라, 정규식 층을 순서대로 벗겨내는 방식으로 충분히 커버된다. 실제로 짜서 돌려 봤다 — 외부의 존성 없이 약 160줄이다.

직접 만든 규칙 기반 파서 — 실행 결과 (기준: 2026-09-02 수 13:30)			
입력제목시작 - 종료부가			
내일 오후 3시 팀 회의	팀 회의	09-03 15:00 - 16:00	—
다음주 화요일 10시 치과 #개인	치과	09-08 10:00 - 11:00	개인
9월 10일 14시~16시 월간보고 #업무 !30분전	월간보고	09-10 14:00 - 16:00	업무 · 30분전
금요일 저녁 7시 저녁약속	저녁약속	09-04 19:00 - 20:00	—
매주 월요일 9시 주간회의 #업무	주간회의	09-07 09:00 - 10:00	매주월요일
모레 종일 워크숍 #학습	워크숍	09-04 종일	학습
3일 뒤 15시30분 계약서 검토	계약서 검토	09-05 15:30 - 16:30	—
2026-09-25 9시 반 건강검진	건강검진	09-25 09:30 - 10:30	—
오늘 16시 1시간 코드리뷰	코드리뷰	09-02 16:00 - 17:00	—
담주 목요일 점심 12시 팀 점심 #개인	팀 점심	09-10 12:00 - 13:00	개인
10/15 종일 출장 #업무	출장	10-15 종일	업무

파서 설계에서 지킬 규칙

- **인식한 구간의 위치(span)를 반드시 함께 반환한다**. 입력창에 밑줄을 긋고 클릭으로 취소하려면 인덱스가 필요하다. 결과값만 넘기면 Todoist식 되돌리기를 나중에 붙일 수 없다.
- **남은 텍스트가 제목이다**. 인식된 구간을 마스킹하고 남은 것을 정리해 제목으로 쓴다. 조사(에·부터·까지·쯤)는 제거한다.
- **모호할 때는 업무 관례를 따른다**. 오전/오후 표시 없는 **3시** 는 15시로 본다. 요일만 말했는데 이미 지났으면 다음 주로 넘긴다.
- **파서를 끌 수 있게 한다**. Todoist에도 "스마트 날짜 인식 끄기"가 있다. "9월 매출 정리"의 "9월"이 날짜로 잡히는 일은 반드시 생긴다.

- 기존 직접 입력을 없애지 않는다. 20260910 · 930 입력은 이미 있는 강점이다. 자연어는 그 위에 얹는 층이지 대체가 아니다.

③ 편집 패널 재설계 — 8필드에서 3필드로

패널을 없애자는 게 아니다. **기본 상태에서 보이는 것을 줄이고**, 클릭한 자리에 뜨는 팝오버 형태로 바꾼다. 지금 폼의 필드 8개 중 **매번** 쓰이는 것은 제목·시간·분류 셋뿐이다.

형태로 바꾼다. 지금 폼의 필드 8개 중 **매번** 쓰이는 것은 제목·시간·분류 셋뿐이다.

바뀐 것 — 캘린더가 가려지지 않고, 드래그한 시간이 곧 기본값이며, 제목 칸에서 자연어가 그대로 먹고, 분류는 실제 색이 칠해진 칩이고, 알림·반복·메모는 +로 필요할 때만 펼쳐지며, 저장 버튼이 흐름의 끝에 있다. 필드 8개 → 기본 3개.

④ 캔버스 직접 조작

바뀐 것 — 캘린더가 가려지지 않고, **드래그한 시간이 곧 기본값**이며, 제목 칸에서 자연어가 그대로 먹고, 분류는 **실제 색이 칠해진 칩**이고, 알림·반복·메모는 **+로 필요할 때만** 펼쳐지며, 저장 버튼이 흐름의 끝에 있다. 필드 8개 → 기본 3개.

④ 캔버스 직접 조작

- **드래그 생성** — 빈 시간을 끌면 그 범위로 팝오버가 열린다. (안내 문구는 지운다. 커서가 `crosshair` 로 바뀌는 것으로 충분하다.)
- **블록 이동** — 일정을 끌어 다른 시간·다른 날로 옮긴다. 월간 뷰에서는 날짜 칸 사이로.
- **모서리 리사이즈** — 아래 모서리를 끌어 길이를 바꾼다. 15분 단위 스냅, Alt를 누르면 1분 단위.
- **되돌리기** — 드래그로 옮긴 뒤 Ctrl+Z. 설명서 9장 원칙 6("되돌릴 수 있게 한다")이 직접 조작에서는 필수 조건이 된다.

PyQt6에서는 `QGraphicsScene` / `QGraphicsItem` 으로 가면 이동·리사이즈·스냅이 대부분 프레임워크 위에서 해결된다. 지금처럼 위젯을 좌표로 배치하는 구조라면 `mousePressEvent` · `mouseMoveEvent` 에서 직접 처리해야 하는데, **이 전환이 4단계 중 가장 큰 구조 변경**이다. 그래서 자연어 입력보다 뒤에 둔다.

⑤ TomaDesk만 할 수 있는 것

여기까지는 다른 앱을 따라잡는 일이다. 앞서가는 지점은 **이미 물려 있는 구조**에서 나온다.

- **일정에서 D-Day로 한 키.** 빠른 일정 창에서 D를 누르면 같은 입력이 D-Day가 된다. 마감 관리가 주 용도인 프로그램에서 이 전환이 매끄러운 앱은 없다.
- **메모를 캘린더로 드래그.** 왼쪽 메모 목록에서 메모를 시간축에 떨어뜨리면 일정이 생기고 원본 메모와 연결된다 — TickTick식 타임 블로킹인데, TomaDesk는 그 항목이 이미 서식·첨부·체크리스트를 가진 메모다.
- **반복 패턴 감지의 확장.** 설명서 7.1의 "매달 포스트잇 제안"을 자연어 입력에도 적용한다. **매주 월요일 9시 주간회의**를 세 번째 손으로 만들면, 반복 일정으로 바꿀지 한 줄로 제안한다.
- **토마팻이 입력 결과를 확인해 준다.** 빠른 일정으로 저장한 뒤 말풍선으로 "9월 8일 10시 치과 · 되돌리기"를 3초간 띄운다. 창을 닫은 뒤에도 무엇이 저장됐는지 보이고, 오인식을 그 자리에서 취소할 수 있다.

05 — 로드맵

순서가 중요하다

아래 순서는 임의가 아니다. 뒤 단계가 앞 단계의 결과물을 쓴다 — 파서는 팝오버 안에서 다시 쓰이고, 팝오버의 데이터 모델이 연동의 매핑 대상이 된다.

011-2주

한국어 파서 + 빠른 일정 창

외부 의존성 없는 규칙 기반 파서를 모듈로 만들고(`ko_schedule_parser.py`), 전역 단축키 Ctrl+Alt+A에 한 줄 입력창을 붙인다. 인식 하이라이트 + 미리보기 + 충돌 검사까지.

- 파서는 **span 포함**으로 반환 — 3단계의 하이라이트가 이것을 쓴다
- pytest 케이스 40건 이상(기존 190건 체계에 추가)
- 설정에 "자연어 인식 사용" 토글

효과 큼위험 낮음기존 구조 변경 없음

021주

기본값·저장 흐름 정리 (현재 폼 유지)

구조를 바꾸기 전에, 지금 폼에서 바로 고칠 수 있는 마찰부터 없앤다. 하루면 체감이 달라지는 항목들이다.

- 클릭한 시간 / 현재 보고 있는 시간을 시작값으로 (19:00 고정 제거)
- 마지막 필드에서 Ctrl+Enter 저장, Esc 닫기
- 분류 드롭다운에 **색 점** 표시 — 캘린더 칩 색과 일치
- 메모 입력란을 접힌 상태로 시작 (+ 메모)
- 상단 안내 문구를 커서 변화·툴팁으로 대체

효과 큼비용 최소

032-3주

인라인 팝오버 + 드래그 생성

고정 사이드 패널을 클릭한 자리의 팝오버로 바꾸고, 시간축 드래그로 범위를 지정한다. 팝오버 제목 칸은 1단계 파서를 그대로 쓴다 — 자연어와 폼이 같은 입력창에서 만난다.

- 기본 3필드(제목·시간·분류) + +로 펼치는 알림·반복·메모·D-Day
- 기존 전체 편집 화면은 "전체 편집 ↗"로 남긴다 — 없애지 않는다
- 15분 스냅, Alt 정밀 모드

효과 큼구조 변경

042-3주

블록 이동·리사이즈 + 되돌리기

만들어진 일정을 끌어 옮기고 늘인다. 일간·주간은 시간축 위에서, 월간은 날짜 칸 사이에서. 직접 조작에는 Ctrl+Z가 반드시 함께 간다.

- QGraphicsScene 전환 여부를 이 단계 착수 전에 결정
- 실행 취소 스택 — 최소 이동·리사이즈·삭제 3종
- 메모 목록 → 캘린더 드래그(타임 블로킹)를 여기에 얹는다

난이도 높음테스트 부담

051주

.ics 내보내기 / 가져오기 (L1)

Excel 내보내기 옆에 붙인다. 선택한 기간·선택한 분류만 내보내기, 가져올 때 중복 검사(UID 기준). 이것만으로 구글·아웃룩·네이버 어디로든 옮길 수 있다.

비용 최소요구 충족률 높음

062-4주

외부 캘린더 구독 → 양방향 (L2 → L3/L4)

먼저 **읽기 전용 미리**로 외부 일정을 회색 블록으로 표시한다(공휴일·회사 캘린더). 그 다음에야 CalDAV 또는 Google/Graph API로 쓰기를 연다. 매핑 테이블·증분 토큰·충돌 정책은 이 단계에서 확정한다.

- 폐쇄망 사용자가 있다면 로컬 Outlook COM(L0)을 L3보다 먼저
- 토큰은 Windows 자격 증명 관리자에 — SQLite 평문 저장 금지

난이도 높음인증·보안 고려

← 구현 비용 낮음구현 비용 높음 →

지금 바로 / 효과 큼

- 클릭한 시간을 기본값으로
- Ctrl+Enter 저장 · Esc 닫기
- 분류에 색 점
- 메모 칸 접기
- .ics 내보내기

계획해서 / 효과 큼

- 한국어 자연어 파서
- 빠른 일정 (Ctrl+Alt+A)
- 인라인 팝오버 + 드래그 생성
- 블록 이동 · 리사이즈

지금 바로 / 효과 보통

- 단축키 오버레이 (?)
- 시간 프리셋 버튼 (30분 · 1시간 · 2시간)
- 안내 문구 → 커서 · 툴팁

나중에

- Google / Graph API 양방향
- 메모 → 캘린더 타임 블로킹
- 커맨드 팔레트
- 자연어로 수정하기

06 — 결정

진행 전에 정해야 할 것

1. **사이드 패널을 팝오버로 바꿀 것인가, 둘 다 둘 것인가.** 넓은 화면에서는 사이드 패널이 편할 수 있다. 설명서 9장 원칙 2(창 크기에 따라 배치가 바뀐다)를 따르면, *좁으면 팝오버 · 넓으면 패널*도 가능하다. 다만 두 경로를 유지하는 비용이 생긴다.
2. **** QGraphicsScene **으로 갈 것인가.** 4단계의 난이도를 결정하는 갈림길이다. 지금 캘린더 구현이 위젯 배치 기반이라면, 전환 자체가 별도 작업이 된다.
3. **자연어 파서의 오인식 허용 범위.** "9월 매출 정리"의 "9월"을 날짜로 잡을 것인가. 안전하게 가려면 *날짜 표현이 문장 앞 부분에 올 때만* 인식하는 식으로 제한할 수 있다.
4. **연동 대상 우선순위.** 실제 사용 환경이 구글인지, 회사 아웃룩인지, 폐쇄망인지에 따라 6단계의 경로가 완전히 갈린다.

참고 자료

- Fantastical[Adding Events and Tasks — Flexibits](#)
- Todoist[Use Task Quick Add in Todoist](#)
- Todoist[Turn smart date recognition on or off](#)
- Notion Calendar[Notion Calendar keyboard shortcuts](#)
- 비교[Calendar Apps That Understand Natural Language \(2026\)](#)
- Vimcal[Vimcal wants to be the most nifty calendar app on the block — TechCrunch](#)
- TickTick[TickTick Calendar Quickstart: 4 Steps to Complete Time Blocking](#)
- Google API[Get push notifications — Google Calendar API](#)
- Microsoft[Get incremental changes to events in a calendar view — Microsoft Graph](#)
- Microsoft[Outlook calendar API overview](#)
- 실측[dateparser 4.x · 규칙 기반 파서 프로토타입 — 이 세션에서 직접 실행](#)